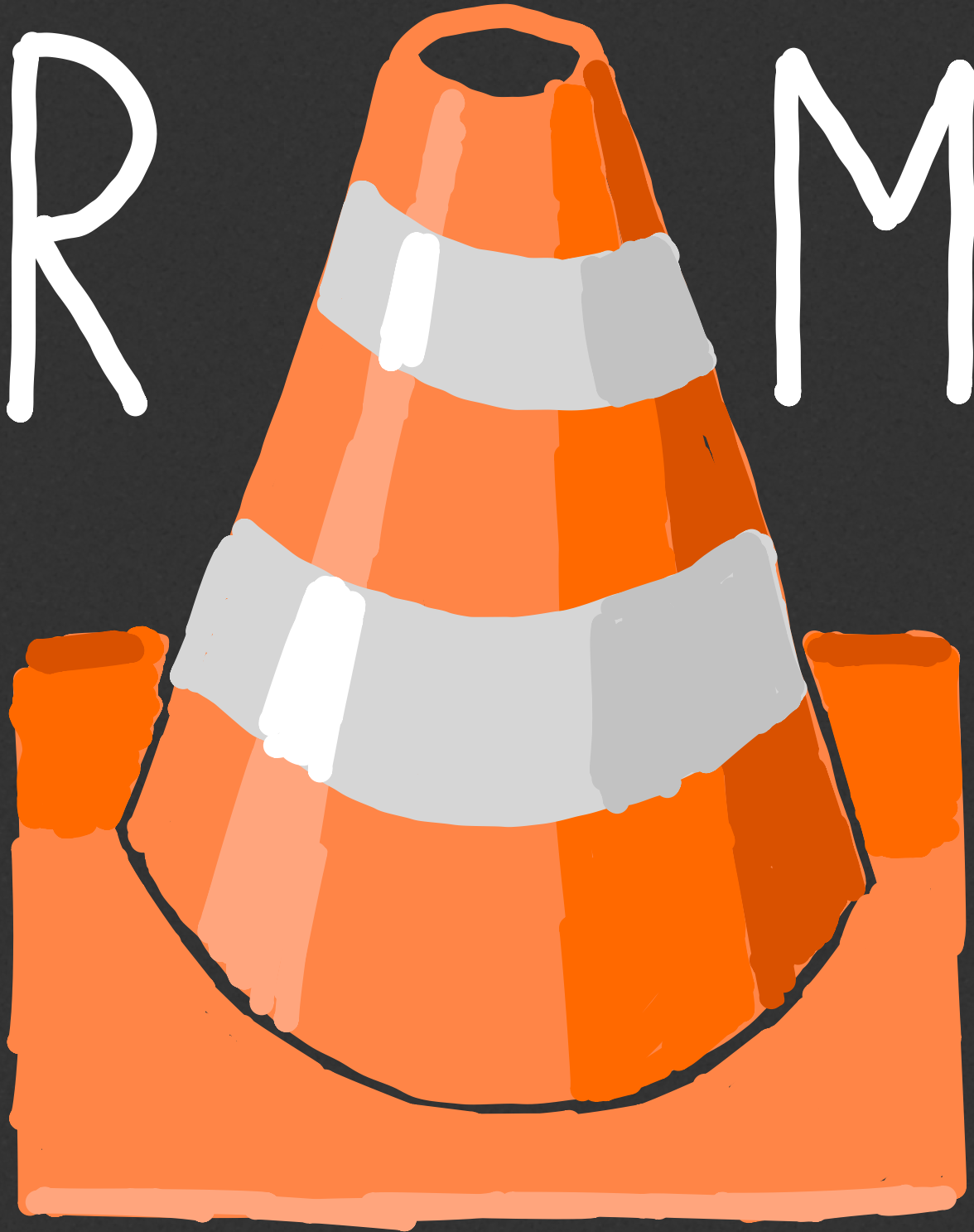


All rights unreserved

(2nd edition)



`rm -rf /bin/life`

END USER LICENSE AGREEMENT

by reading the following agreement you automatically agree that: You owe the author of this work a blow job
xoxo ♡

Copyright© ME BITCH

Diskretna valčna transformacija (DWT)

- Pri DCT slaba časovna resolucija, problem pri: večnem stiskanju (vidijo se bloki)
- Valčna transformacija, dobra frekvenčna in časovna redukcija

DWT

- CWT in DWT predstavitev signala v frekvencah na različnih prostorskih (časovnih) resolucijah
 - Višje frek.: krajša okna → dolga prostorska ločljivost, slaba frekvenčna
 - Nižje frek.: širše okna → slaba prostorska ločljivost, dobra frekvenčna
 - Osnovna ideja DWT pri zap. številah
 - različno velika okna (krajša/daljša prostorska ločljivost)
 - Analiza visokih frekvenc: Analiza frekvenc med nekaj sosednjimi vrednostmi
 - Analiza nižjih frek.: Analiza razlik znotraj daljšega zaporedja.
- Prisotne so visoke frekvence v daljših zaporedjih (Rešitev: odstranimo, analiziramo preostanek)

Diskretna Haarova valčna transformacija

- Analiza sosednjih vrednosti v vsakem paru vrednosti
- Rekurziven postopek:
 - x_i predstavimo z x_{j+1} in razlikami d_{j+1}

$$x_{j+1,i} = \frac{x_{j,2i} + x_{j,2i+1}}{2}$$

$$d_{j+1,i} = \frac{x_{j,2i} - x_{j,2i+1}}{2}$$

Razširitev za 2D (naivno)

- Prvo izvedemo DWT nad vrsticami v celoti, nato DWT nad stolpci v celoti

$$x_{j,2i+1} = x_{j+1,i} - d_{j+1,i}$$

$$x_{j,2i} = x_{j+1,i} + d_{j+1,i}$$

Razširitev za 2D (pogost pristop)

- piramidna valčna dekompozicija
 - 1. izvedba nivoja DWT nad vrsticami,
 - 2. izvedba DWT nad stolpci (po zgornji formuli)
 - Ponovimo postopek nad LL

1.		2.	
L	H	LL	HL
		LH	HH

Matrični zapis Haarove transformacije

- $x_0 = \{10, 13, 25, 26\}$
- Da izračunam $\{x_1, d_1\} = W_1 x_0$
- $\{x_2, d_2, d_1\} = W_2 W_1 x_0$
- Matrika H je Haarova matrika brez skalirnih faktorjev

$$H_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \quad H_{2N} = \frac{1}{\sqrt{2}} \begin{bmatrix} H_N \otimes [1 \ 1] \\ I_N \otimes [1 \ -1] \end{bmatrix}$$

↑ Kronekerjev produkt
↑ Identiteta za $N \times N$

$$W_1 = \frac{1}{2} \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & -1 \end{bmatrix}$$

$$W_2 = \frac{1}{2} \begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$W = \begin{bmatrix} 1/4 & 1/4 & 1/4 & 1/4 \\ 1/4 & 1/4 & -1/4 & -1/4 \\ 1/2 & -1/2 & 0 & 0 \\ 0 & 0 & 1/2 & -1/2 \end{bmatrix}$$

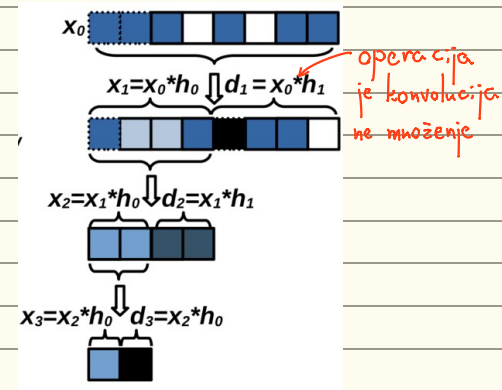
- Veliko 0 v matrikah redundanca
- Visoka časovna zahtevnost $O(n^2)$
- Večinoma boljše način, neposreden izračun

DWT - splošno

- Z valčkom je možno $x_j \rightarrow \{x_{j+1}, d_{j+1}\}$
- Uporaba dveh valčnih funkcij:
 - osnovna: $d_{j+1}: \Psi$
 - skalirna funkcija za aproksimacijo $x_{j+1}: \Phi$
- uporaba skalirne funkcije za izvedbo Ψ z navidezno daljšim oknom (Ψ izvajamo nad vedno bolj grobim signalom)
 - $x_j \rightarrow \{x_{j+1}, d_{j+1}\}$
 - $x_j \rightarrow \{\{x_{j+2}, d_{j+2}\}, d_{j+2}\}$

Predstavitev DWT s filtri

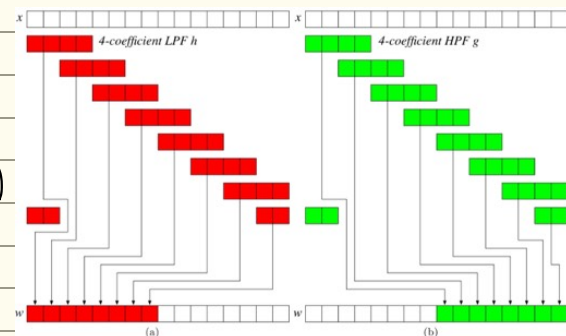
- x_{j+1} - nižje frekvence; d_{j+1} - visoke frekvence / podrobnosti
- namesto Ψ in Φ uporabimo filtra h_0 in h_1
- filtriranje - linearna operacija, ki se izvede s konvolucijo
 - Haarov filter: $\Psi - h_1 = [0.707, -0.707]$
- Uporaba: $x_{j+1} = x_j * h_0$
 $d_{j+1} = x_j * h_1$
- Težava: $|x_{j+1}| \geq |x_j|$ in $|d_{j+1}| \geq |x_j|$
 - neugodna za stiskanje želimo $|x_{j+1}| + |d_{j+1}| = 2|x_j|$
- Rešitev: podvzorčimo ($\downarrow 2$ - vsaka druga vrednost x_j se zavrže)
- Za vsak naslednji koeficient DWT pomaknemo filter za 2 mesti naprej.



npr.:
 $y[0] = x[-2]h[2] + x[-1]h[1] + x[0]h[0]$
 $y[1] = x[0]h[2] + x[1]h[1] + x[2]h[0]$
:

- Možna rekonstrukcija z IDWT
 - Uporaba nadvzorčaja, med vrednostmi v x in d vrzimo 0 (vsak sodi element = 0)

$$x_{j+1} = [(x_j \uparrow 2) \cdot g_0 + (d_j \uparrow 2) \cdot g_1]$$



Implementacija DWT s filtri

- Razmerje med h_1 in h_0 : $h_1[n] = (-1)^n h_0[1-n]$
- Za Haarovo transformacijo:
 $h_0 = [0.707, 0.707]$ $h_1 = [0.707, -0.707]$
- Inverzni DWT, če so valčki h ortogonalni, so sintezni valčki g samo obrnjeni valčki h ($g[n] = s[-n]$)

$$x_{j+1}[n] = \sum_k (x_j[k]g_0[n-2k]) + \sum_k (d_j[k]g_1[n-2k])$$

- Zgornja izpeljava velja samo za normalizirane valčke.
Za ostale so kompleksnejši postopki

- Ker je zapis za konvolucijo splošen je treba določiti kaj pomeni npr. -1 :
 - Si predstavljamo, da je krožno zaporedje $x[-1]$ = zadnji element
 - Element predstavimo s 0
- Vrstni red koeficientov filtra se pred konvolucijo pogosto obrne glede na koeficiente filtra:
 - Filter dolžine 2: $d_{i+1}[0] = x_i[0] \cdot h_1[0] + x_i[1] \cdot h_1[1] = x_i[0] \cdot 0.707 + x_i[1] \cdot (-0.707)$
 - Filter dolžine 4: $d_{i+1}[0] = x_i[-1] \cdot h_1[0] + x_i[0] \cdot h_1[1] + x_i[1] \cdot h_1[2] + x_i[2] \cdot h_1[3]$
- Primeri valčkov:

TABLE 8.2: Orthogonal wavelet filters

Wavelet	Number of taps	Start index	Coefficients	
Haar	2	0	[0.707, 0.707]	db1
Daubechies 4	4	0	[0.483, 0.837, 0.224, -0.129]	db2
Daubechies 6	6	0	[0.332, 0.807, 0.460, -0.135, -0.085, 0.0352]	db3
Daubechies 8	8	0	[0.230, 0.715, 0.631, -0.028, -0.187, 0.031, 0.033, -0.011]	db4

Časovna zahtevnost

- DCT: $O(n^2) \rightarrow$ FDCT: $O(n \cdot \log n)$
- DWT: $T(n) = 2n + T(n/2) = O(n)$
- Za DWT lažje uporabljamo večje bloke

Uporabljeno

Slike: JPEG 2000, JPEG LS

Video:

- MJ2 - Motion JPEG 2000
- Dirac (odprava redundance znotraj slike DWT-intra frame)
(odprava redundance med okviri - inter frame)
- SMPTE VC-2 (za obdelavo videa)

JPEG 2000 - razlike z JPEG

- počasnejše delovanje
- patent;
- uporaba v medicini (večje dimenzije slik in več kanalov)

JPEG 2000 postopek stiskanja

1. pretvorba v YCbCr ali YUV (za lossless)

2. ploščice

- razdelimo sliko na ploščice (so lahko dosti večje kot pri JPEG)
- lahko ena sama
- ploščice lahko procesiramo neodvisno ene od druge
- naključen dostop do različnih delov slike

3. Uporaba DWT za frekvenčno analizo

- uporaba biortogonalnih valčkov za izgubno stiskanje
- Možna tudi pri brezizgubnem stiskanju

4. kvantizacija, vrednosti iz DWT zaokrožimo

5. Zapis vrednosti z EBCOT, za napredujoče stiskanje

6. Kodiranje bitov s kontekstno odvisnim adaptivnim aritmetičnim kodirnikom

- Kodiranje se prilagaja gledena okolico slike, kodiramo na njenoj pikslu

JPEG XS

- odstranjevanje ne stisnjenih slik tudi na šibkejših napravah (niska prostorska in časovna zahtevnost)
- Strojna implementacija (ASIC, FPGA, GPU)
- Omogoča vizualno brezsegurno stiskanje (1:10)
- Lahko uporabljamo 12 bitne barve
- Ideja: uporaba delne valčne transformacije

Kodiranje koeficientov DWT

- Najprej zapišemo aproksimacijo nato podrobnosti (napredujoče stiskanje) z DWT dobimo napredujoče stiskanje ločljivosti ne kakovosti
- Boljši pristop EZW, vgrajeno drevo nižel valčkov
- EZW omogoča napredujoče stiskanje po kakovosti
 - Kodirnih pri kodiranju ustvari dve različno veliki datoteki; začetek večje datoteke je enak manjši datoteki
- EZW je temelj za napredujoče stiskanje koeficientov DWT
 - Bolj napredne metode: SPIHT, EBCOT

EZW (embedded zerotree wavelet) ideja

- Predpostavka: če so koeficienti nižjih frekvenc manjši, potem so tudi manjši koeficienti višjih frekvenc v istem delu slike (predpostavimo, da so vrednosti koeficientov podrobnosti manjši od koeficientov aproksimacij)
- Predpostavimo abs. večje vrednosti so pomembnejše za vizualno zaznavo.

EZW ideja za kodiranje

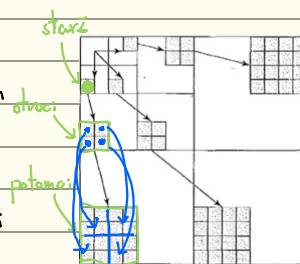
- Prvo obravnavamo nižje nato višje frekvence
- Kodiranje podobno kot pri bitnih ravneh (progresivno kodiramo najpomembnejše informacije koeficientov)

EZW gradniki: kodirnika

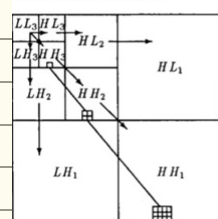
- Drevesa nižel
- Zaporedna aproksimacija kvantizacije, kodiramo koeficiente po vzoru kodiranja bitnih ravnin (kodiranje ni identično, ker EZW podpira decimalke)

EZW drevo nižel (zero tree)

- koeficiente predstavimo z štiriskim drevesom (k : je prostorsko orientiran (i?))
 - **Star** - koeficient nižje frekvence
 - **Otrok** - koeficienti višjih frekvenc, ki so podrobnosti istega področja z enako orientacijo
 - **Potenci** - vsi koeficienti, ki predstavljajo coeffs. višjih frekvenc (podrobnosti), ki predstavljajo isto področje in orientacijo od prednikov



IRE 8.22: Parent-child relationship in a zero tree
Dero, M. S., & Liu, J. (2004). Fundamentals of multimedia (pp. 148-149). Pearson Education.



- Ine EZW izhaja iz načina kodiranja; če je vrednost bita koeficienta 0, sklepamo, da so ostali potamci tudi 0 (takrat informacijo kodiramo s posebnim simbolom in bite potamcev ne kodiramo)

- Def. kodiranja:

1. možna stanja koeficientov

- če je $|x| < T$, je x nepomemben (x = koeficient),
 - če so vsi potamci $|y| < T$ (nepomembni), je stanje od x : koren ("t")
 - če obstaja potomec $|y| \geq T$, je stanje od x : izoliran ("y")
 - če je x list ga označimo z "t"
- če je $x \geq T$, je x pomemben in pozitiven, stanje od x : pozitiven in signifikanten ("p")
- če je $x \leq -T$, je x pomemben in negativen, stanje od x : negativen in signifikanten ("n")

2. zaporedna aproksimacija kvantizacije

- koeficiente kodiramo od nižjih do višjih frekvenc
- kodiramo po vzoru bitnih ravni (basically MSB $\rightarrow$ LSB)

npr. $x = 63$:

- najprej kodiramo najpomembnejšo informacijo: je $x > 32$
- kasneje kodiramo manj pomembne informacije (dejansko kodiramo v več prehodih, z vsakim prehodom zajemamo več in več podrobnosti)

$x > 48, x > 56$

Primer:

$x = 58$, izberemo največji možni prag T , kjer $T = 2^y \wedge |x| > T$

$T = 32 \Rightarrow$ zapišemo T na izhod

- določimo stanje coeff: "n" ali "p"

$x \geq T \Rightarrow$ na izhod pošljemo "p"

$x < -T \Rightarrow$ na izhod pošljemo "n" z do sedaj kodiranim vemo, da je $x \in [32, 64)$, ter predznak

- izvedeno podrejene korake, kjer interval razpolavljamo in glede na pozicijo št. v intervalu (zgoraj, spodaj) na izhod izpišemo 0 (spodaj) ali 1 (zgoraj)

1. $[32, 64)$, $58 > 48 \Rightarrow [48, 64) \Rightarrow$ izhod: 1

2. $[48, 64)$, $58 > 56 \Rightarrow [56, 64) \Rightarrow$ izhod: 1

3. $[56, 64)$, $58 < 60 \Rightarrow [56, 60) \Rightarrow$ izhod: 0

...

- to lahko delamo do poljubne stopnje detajla (decimalka, $SSIM < 0,95$)

EZW vrstni red prebiranja koeficientov

- Najprej obravnavamo koeficiente nižjih frekvenc, nato že višje lahko (obiskujemo po z krivulji)
- Smiselno se kodirati koef. od najvišje do najmanjše vrednosti (bolj učinkovito napredujoče stiskanje)

- Postopek:

• Izbere se T tako da $T = 2^y \wedge T \geq \max(|x|)$

• $LS = \{\}$, seznam koef. x za podrejen korak

• dokler ne dosežemo določene kakovosti (npr. $SSIM < 0,95$)

1. Prevladujoč korak: na izhod zapišemo stanje koef. x ("n", "p", "t", "y")
v LS zapišemo $|x|$

2. Podrejen korak / izboljšava: izpišemo 1 ali 0 glede na kje v intervalu je x

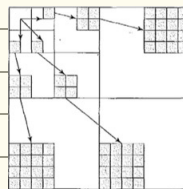
3. $T = T/2$ repeat

$\uparrow$ to je enako kot zgoraj

57	37	39	20	3	7	9	10
29	30	14	33	8	2	1	6
14	6	15	13	9	4	2	3
16	39	4	9	7	14	12	-9
12	15	33	20	-2	3	1	0
0	7	2	4	4	-1	1	1
4	1	10	3	2	0	1	0
5	6	0	0	3	1	2	1

Prevladujoč korak (bolj podrobno)

- Stanja ("n", "p", "t", "y")
- Pri stanju "t" preskočimo vse potence
- Koeficiente "p" in "n" vstavi v LS kot $|x|$
- Koeficienti, ki so že v LS iz prejšnjega prehoda postavi na 0



Podrejen korak (bolj podrobno)

- Delamo nad koef. v LS
- Z vsaki koeficient izpišemo 0 ali 1 (označimo interval)
- Po izvedbi podrejenega intervala izvedemo ožanje intervala uredimo LS padajoče, na podlagi informacij dostopnih dekodirniku.

Detali: kodiranja EZW

- Izhodi: prag T, simboli, zaporedje bitov iz izboljšav
- za simbole uporabljamo entropijski kodirnik
- v LS je smiselno voditi tudi koordinate koeficientov

EZW aplikacije

- Možna uporaba ROI za: drugačno stopnjo stiskanja, napredujoče stiskanje
- Vrednosti v ROI povečamo (naredimo bolj pomembne)
- podobne ideje v JPEG2000: EBCOT

SPIHT (set partition in hierarchical trees)

- Nadgradnja nad EZW, bolj optimalno vgrajeno kodiranje, delo s celimi števili, bolj učinkovito bolj kompleksno, zelo učinkovito že brez uporabe aritmetičnega kodirnika

SPIHT osnovna ideja

1. Init: zapis n ($T = 2^n$, podobno kot pri EZW), $n = \lfloor \log_2(\max_{i,j} \{|c_{i,j}|\}) \rfloor$ $c_{i,j}$ koeficient
2. Urojanje, zapis koordinat $c_{i,j}$, ki so pomembni ($2^n \leq |c_{i,j}| < 2^{n+1}$) in predznahe glede na optimalen vrstni red (urejanje na podlagi razdelitve množice)
3. Izboljšava: izpis ntega MSBja $c_{i,j}$, kjer $|c_{i,j}| \geq 2^{n+1}$, vsi $c_{i,j}$ zapisani v prejšnji iteraciji
4. $n = n + 1 \rightarrow$ korak 2.

SPIHT terminologija

- Zero tree iz EZW $\rightarrow$ Spatial orientation trees (SOT)
- $c_{i,j}$ na koordinati (i,j) je koren drevesa
- $O(i,j)$ je množica koeficientov, ki so otroci $c_{i,j}$
 - koef. višjih podrobnost, ki predstavljajo isto področje slike kot (i,j)
 - $O(i,j)$ so na nižjem nivoju SOT
 - $c_{i,j}$ ima 4 sinove ali na benega če je list
 - ~~Apromksimacija~~ ~~hima~~ sinov
 - Hierhija: razdelimo množice $O(i,j)$, dobimo 4 nove $1 \rightarrow 2 \times 2$

- $D(i,j)$ - množica naslednikov, koef, ki so potomci koef. c_{ij}
 - aproksimacija nima naslednikov
- $L(i,j)$ - množica potomcev brez otroke $D(i,j) - O(i,j)$
- Primer:
 - $O(0,0) = \{ \}$ ← aproksimacija
 - $O(1,0) = \{ (2,0), (2,1), (3,0), (3,1) \}$
 - $D(1,0) = \{ (2,0), (2,1), (3,0), (3,1), (4,0), \dots, (7,3) \}$

- Uporaba treh seznamov

- LSP - koef. oz. koordinate, pikslor, ki so pomembni $|c_{ij}| > T$
- LIP - koef. -li-, ki niso pomembni ($|c_{ij}| < T$), predvideni za poznejše testiranje signifikantnosti
- LIS - koren oz. koordinate, kjer so vsi elementi v $D(i,j) < T$ (jih še nismo kodirali)

- Funkcija signifikantnosti: S

- $S(i,j) = 1$, če $|c_{ij}| > T$, sicer $S(i,j) = 0$
- $S(D(i,j)) = 1$, če $\exists (i,j) \in D(i,j) : |c_{ij}| > T$
- $S(L(i,j)) = 1$, če $\exists (i,j) \in L(i,j) : |c_{ij}| > T$

$$S_n(T) = \begin{cases} 1, & \max_{(i,j) \in \mathcal{I}} |c_{ij}| > 2^n \\ 0, & \text{sicer} \end{cases}$$

SPIHT algoritem

- Inicializacija:

$$LSP = \{ \}$$

$$LIP = \{ \text{koef. najvišjega nivoja DWT kot koordinate} \}$$

$$LIS = \{ \text{elementi iz LIP, } D(i,j) \neq (0,0) \}$$

$$T = 2^n; n = \lfloor \log_2(\max \{ |c_{ij}| \}) \rfloor$$

- Izvajanje dokler $n \geq 0$

- Korak urejanja

- obdelaj se LIP (polnimo LSP ⇒ izpišuje se 0/1)

- obdelaj se LIS (polnimo LSP, LIP, LIS ⇒ izpišujemo 0/1)

- Izboljšave; vsaki c_{ij} v LSP razen el. trenutne iteracije:
 - zapiši na izhod n -ti bit $|c_{ij}|$

$$T = T/2, n = n-1$$

Obdelava LIP

- Za vsako c_{ij} v LIP če je $S(i,j) = 1$ ⇒ premakni (i,j) iz LIP v LSP

Urejanje

- za vsako množico D/L v LIS

- če je množica $D(i,j)$, izpiši $S(D(i,j))$ in če $S(D(i,j)) = 1$ odstrani $D(i,j)$ iz LIS. Vsakega otroka $(h,l) \in O(i,j)$ na izhod izpišemo $S(h,l)$, če je =1 vstavi v LSP in izpiši predznak c_{hl} , če je =0 vstavi (h,l) v LIP, če je množica $L(i,j) > 0$, vstavi $L(i,j)$ na konec LIS

- če je množica $L(i,j)$
 - zapiši $S(L(i,j)) = 1$, vstavi vsakega otroka $(k,l) \in O(i,j)$ na konec LIS kot $D(k,l)$, ter odstranimo (i,j) iz LIS

Krivulje polnenja prostora

Theora video kodiranje

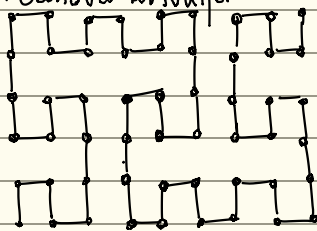
- Posamezne okvirje kodiramo podobno kot pri JPEG DCT, napoved DC med sosednjimi bloki
- Tipi okvirjev:
 - I - zapisani kot original slika (golden frame, intrahodiranje)
 - P - zapisano kot razlike glede na prejšnji (referenčni) okvir (interframe kodiranje)
- Z DCT kodiramo I in P okvirje
- I in P tudi osnove drugih formatov MPEG, H.263, H.265...
- Izboljšava združimo 8×8 v 16×16 , makrobloke napovedujemo z uporabo vektorjev gibanja, v makrobloku zapišemo vektor gibanja, ter blok napovedemo z referenčnim (ga odštejemo od referenčnega)
- razlike kodiramo z DCT
- Želimo napovedati koeficiente DC skeniranja pretvorba nina najbolj optimalna, uporabimo hilbertovo krivuljo za prebiranje makroblokov
- združujemo makrobloke v superbloke 2×2 , kodiranje blokov znotraj superblokov z Hilbert krivuljo (učinkovitajšo napovedovanje DC koef.)

Krivulje polnjenja prostora

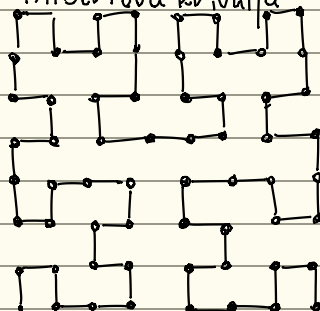
- Preslikava več dimenzionalnih podatkov v zaporedje z uporabo prostorskega indeksa (krivulje polnenja prostora), preslikave prostorskih koordinat v indekse

- Primeri

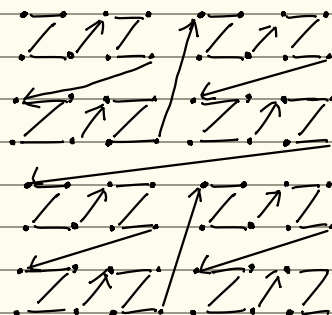
• Peanova krivulja



• Hilbertova krivulja



• Z-krivulja enako kot pri EZW



Problem z to je ker se vedno potrebuje večje skoke med sosednjimi vrednostmi

Paše v hilbertov hotel?

~\(\cup\)/~

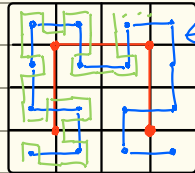
- Red krivulje št. korakov konstrukcije

Hilbertova krivulja (2D)

- Verzija končnega reda je aproksimacija krivulje polnega prostora, ker ne pokriva polnega prostora
- krivuljo reda k lahko konstruiramo gleda na krivuljo $k-1$, krivulja reda k sestoji iz 4 reda $k-1$

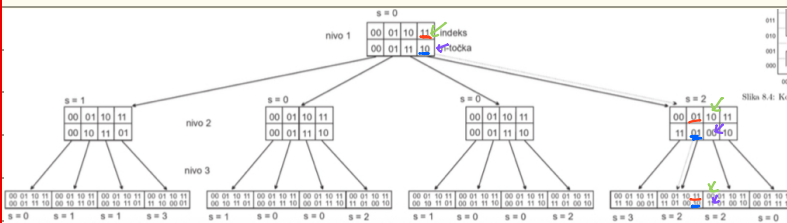
- Konstrukcija v ravnini:

- indeksi: $0, 1, 2, 3 \Rightarrow 00, 01, 10, 11$ (koordinate, so dejansko grayev kode xy)
- za drugi red del delimo na 4 dele na x in y osi: so vrednosti: $00, 01, 10, 11$



Predstavitel Hilbertove krivulje z drevesom

- Indeksi - zap. št. celice na krivulji (lokalno)
 - n -točka - xy coord na krivulji (lokalno)
- Primer za 3 nivoje



Primer 1:

$$x=5 \Rightarrow 101$$

$$y=2 \Rightarrow 010$$

$$\text{indeks: } 110111 = 55$$

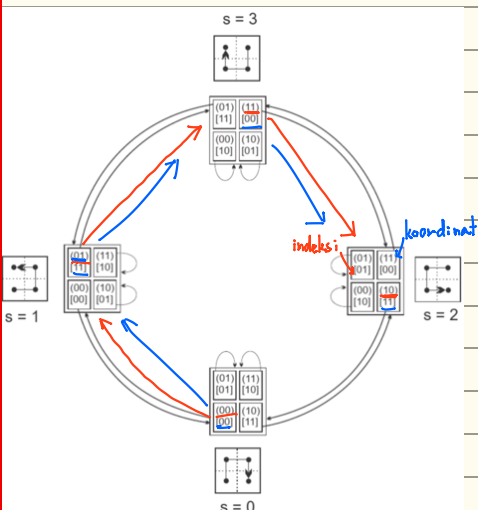
Primer 2:

$$\text{indeks: } 56 \Rightarrow 111000$$

$$x=5 \quad y=1$$

- Pravila za nadaljno deljenje četrtin za otroke glede na starše
- 1. četrt zamenjamo x in y
- 2. in 3. četrt ostane tak h oje
- 4. četrt zamenjamo x in y in negiramo

Transformacije z diagramom stanj



Primer 1:

$$x=3; y=6$$

$$00 \rightarrow 01 \rightarrow 11 \rightarrow 10$$

$$00 \rightarrow 11 \rightarrow 10 \rightarrow 01 = 00110011 = 51$$

začnemo vedno v stanju $s=0$

Primer 2:

$$\text{ind: } 51$$

$$00 \rightarrow 11 \rightarrow 00 \rightarrow 11$$

$$x=3$$

$$y=6$$

$$x \quad y$$

Druge aplikacije Hilbertove krivulje

- v 3D, stiskanje vektorskih podatkov

Fraktali;

- Geometrijske oblike, ki imajo podrobnosti na poljubnih manjših skalah (vzorci, ki so na manjših skalah podobni sami sebi)
- Naravni objekti: dreve, veje, strde, oblaki

Dimenzija

- Dimenzija prostora $\neq$ dim. objekta
- Dim. prostora $\Rightarrow$ prostorska stopnja, celo št.
- Dim. geom. objekta $\Rightarrow$ dimenzija ni vedno celoštevilska

Definicije

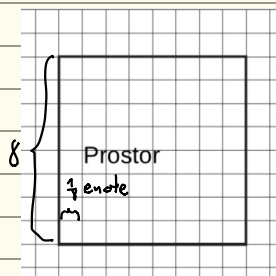
- D-dimenzija
- ϵ - dolžina stranice celic s katerimi želimo zapolniti prostor objekta
- N - št celic v objektu

$$N(\epsilon) = \left(\frac{1}{\epsilon}\right)^D \quad \text{za kvadrat}$$

Primer:

$$D=2 \quad \epsilon = \frac{1}{8}$$

$$N(\epsilon) = \left(\frac{1}{\frac{1}{8}}\right)^2 = 8^2 = 64$$



Splošen zapis formule

$$N(\epsilon) = k \left(\frac{1}{\epsilon}\right)^D \quad \text{pri trikotniku je } k = \frac{1}{2} \text{ npr.}$$

Formula za izračun dimenzije objekta

$$D = \frac{\ln N(\epsilon)}{\ln \frac{1}{\epsilon}} \quad D \text{ ni nujno da je celo število}$$

Primer:

$$\epsilon = \frac{1}{8} \\ N = 64$$

$$\frac{\ln 64}{\ln 8} = 2 \quad \frac{\ln (2^6)}{\ln 2^3} = 2$$

$$\epsilon = \frac{1}{16} \\ N = 32$$

$$\epsilon = \frac{1}{1024} \\ N = \frac{1024^2}{2}$$

$$\frac{\ln 32}{\ln 16} = 1\frac{2}{3}$$

$$\frac{\ln \frac{1024^2}{2}}{\ln 1024} = 1,9$$

- Kadrc zmanjšujemo ϵ povečujemo št. zasedenih celic v objektu, kar povzroča blizanje dejanski dimenziji objekta za list npr. med 1 in 2
- Fraktalni objekti imajo ne celoštevilske dimenzije pri $\epsilon \rightarrow 0$ (epsilon se približuje 0)

Primer izračun dolžine obale:

$$D = 1,2$$

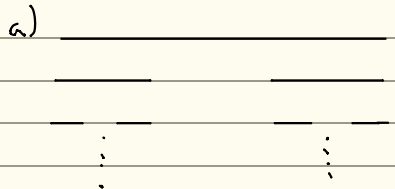
$$\epsilon = \frac{1}{70}$$

$$N(\epsilon) = \left(\frac{1}{\frac{1}{70}}\right)^{1,2} = 70^{1,2} = 163,73 \cdot 1 \text{ km} = 163,73 \text{ km}$$

Tvorba fraktalov

- želimo tvoriti kompleksnejšo obliko, kjer je vidna samopodobnost
- Pogoste tehnike: iterativni funkcijski sistem, uporaba dinamičnih sistemov, L-sistemi
- Primer za cantorjevo množico razdelitev na 3 enake dele odstranimo srednjega, točke vidne ko gremo $\rightarrow \infty$

a)


$$D = \frac{\ln 2}{\ln \frac{1}{3}} = 0,631$$
$$D = \frac{\ln 4}{\ln \frac{1}{3}} = 0,631$$

Tvorba fraktalov na podlagi dinamičnih sistemov

- nova vrednost na podlagi prejšnje

točka: $x_{n+1} = f(x_n)$

z vektorji: $\begin{bmatrix} x_{n+1} \\ y_{n+1} \end{bmatrix} = f \begin{bmatrix} x_n \\ y_n \end{bmatrix}$

s kompleksnimi št.: $z_{n+1} = f(z_n)$

$$z_n = z_{\text{real}} + iz_{\text{im}}$$

Mandelbrotova in Juliajeva množica

- Funkcija: $z_{n+1} = z_n^2 + c$, z in c sta kompleksni števili
- Prostor vrednosti: z ; prostor parametrov: c
- odvisno od parametra c lahko funkcija konvergira ali divergira
 - Množica vrednosti c , ki ohranja z pri $z_1 = 0$ omejen, je mandelbrotova množica
 - Množica začetnih vrednosti z_1 , ki ohranja z omejen pri c se imenuje Juliajeva množica
 - $Z_c \rightarrow \forall c \in \text{Mandelbrot}, \exists z_1 \in \text{Julijev}$

Postopek izračuna za Juliajevo množico

- Izračunaj koordinato z , glede na položaj piksela xy , c je parameter npr. $-0.5125 + i 0.213i$, izvedemo rekurzivno enačbo z ; dokler je znotraj praga do maksimalnega št. iteracij
- Barva predstavlja št. izvedenih iteracij
- v Mandelbrotovi množici je neskončno točk vsaka ima svojo Juliajevo množico

L-sistemi

- Uporaba pravil prepisovanja nizov npr.:

Začetni niz: A

Pravila: (A $\rightarrow$ AB), (B $\rightarrow$ A)

$n=0$: A $n=2$: ABA

$n=1$: AB $n=3$: ABAAB

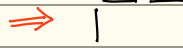
- Lahko se določijo pravila za prikaz določenih simbolov, najpogosteje se uporabljajo zelje grafike(?)

- Primer:

• A-izriži; ravno črta

• R-zavij; desno

• niz: ARAA



- Primer Hilbertova krivulja

• Abeceda: A, B

• Operacije: F, -, + (črta naprej, levo, desno)

• Začetek: A

• Pravila:

- A $\rightarrow$ -BF+AFA+FB-

- B $\rightarrow$ +AF-BFB-FA+

Vidno kako se razdeli na četrtine

Nadgradnja

- Stohastični L-sistem, pravila se aplicirajo z določeno vrjetnostjo

- Koteskno odvisen L-sistem; tehnološki med vejami

- te nadgradnje omogočajo zelo prepričljivo tiskanje dreves

Fraktalno stiskanje

- Uporaba samopodobnosti, opis slike s drugimi deli slike, tako, da najbolj opišejo celotno sliko, uporaba transformacij

- Fraktalna predstavitev slike:

• Transformacije

• Začetna groba slika (domenski blok)

- Rekurzivna izvedba

Ideja dekodiranja

1. nastavi: domenske bloke na določeno obliko

2. transformacije nad bloki da ciljne bloke območja

- bloki območja aproksimirajo org. sliko

3. Vzpostavi: domenske bloke na podlagi:

ciljnih blokov območja

4. Korak 2

- Slika postaja vedno bolj podobna originalni

Ideja kodiranja

1. razdelitev slike na ciljne bloke območja

2. Iskanje domenskih blokov (ki najbolj opišejo ciljne bloke območja)

3. Iskanje transformacij, ki videt domenskih blokov približajo ciljnim blokom območja

4. Rekurzija: ponovi postopek nad domenskimi bloki

- Problem: počasno delovanje (ogromen iskalni prostor)